

On algorithms for strongly-connected components

Tarjan's algorithm, pseudocode

The code originates in *Depth-First Search and Linear Graph Algorithms* Tarjan and is given in a slightly modernized form.

```
function tarjanSCC(V, E):
    i := 0
    S := []
    output := []
    index := {undefined for v in V}
    lowlink := {undefined for v in V}
    onstack := {false for v in V}
    for v in V:
        if index[v] is undefined:
            tarjanSCCstep(v)

    function tarjanSCCstep(v):
        index[v] := i
        lowlink[v] := i
        i := i + 1
        S.push(v)
        onstack(v) := true

        for w in E[v]:
            if index[w] is undefined:
                tarjanSCCstep(w)
                lowlink[v] := min(lowlink[v], lowlink[w])
            else if onstack[w] then
                lowlink[v] := min(lowlink[v], index[w])

        if lowlink[v] = index[v]:
            scc = []
            while index[S.top] ≥ index[v]:
                onstack[S.top] := false
                scc.push(S.top)
                S.pop()
            output.push(scc)

    return output
```

Observations:

1. This is a stateful algorithm with 6 pieces of state
2. `lowlink[v]` is only ever written by the call `tarjanSCCstep(v)`. It is only ever read outside the local scope of `tarjanSCCstep(v)` just after the recursive call has been made. Thus, we can remove this as a global variable and just use ordinary return values to propagate it.

```
function tarjanSCCstep(v):
    ...
    lowlink := i
    ...

    for w in E[v]:
        if index[w] is undefined:
            lowlink_w := tarjanSCCstep(w)
            lowlink := min(lowlink, lowlink_w)
```

```

        else if onstack[w] then
            lowlink := min(lowlink, index[w])

    if lowlink = index[v]:
        ...

    return lowlink

```

3. Each node v can be in one of the following states between calls to `tarjanSCCstep`:

- a. `index[v]` is undefined, `onstack[v]` is false
- b. `index[v]` is defined, `onstack[v]` is true
- c. `index[v]` is defined, `onstack[v]` is false. In this case, we will never read the index of v .

We can represent this with the following ADT:

```

data Status
  = Unvisited
  | Onstack(index)
  | Complete

```

Updating the code:

```

function tarjanSCC(V, E):
    i := 0
    S := []
    output := []
    status = {Unvisited for v in V}
    for v in V:
        if status[v] = Unvisited:
            tarjanSCCstep(v)

function tarjanSCCstep(v):
    status[v] = Onstack(i)
    lowlink := i
    # keep old index around
    iv := i
    i := i + 1
    S.push(v)

    for w in E[v]:
        case status[w]:
            Unvisited:
                lowlink := min(lowlink, tarjanSCCstep(w))
            Onstack(i):
                lowlink := min(lowlink, i)
            Completed:
                # do nothing

    if lowlink = iv:
        scc = []
        while let Onstack(j) = status[S.top] and j ≥ iv:
            status[S.top] := Completed
            scc.push(S.top)
            S.pop()
        output.push(scc)
    return lowlink

return output

```

Now, rather than a for loop, we can transition to a stateful folds in the first loop in the body of `tarjanSCCstep`

```
function tarjanSCCstep(v):
  ...
  lowlink = foldl min iv [
    case status[w] of
      Unvisited → tarjanSCCstep(w)
      Onstack(i) → i
      Completed → ∞
    | w in E[v]]
```

This eliminates `lowlink` as even a piece of local state.

Eliminating **S** and output

`S` mirrors the call stack, except that it might have “extra” nodes in between the call stack nodes that have not yet been popped, as they are waiting to return to an earlier node to become part of the SCC.

Instead of having a global stack, we can pass them upwards as return values.

The same thing can be done with the output: we concatenate the SCCs from child calls and pass this as a result upwards.

```
function tarjanSCC(V, E):
  i := 0
  status = {Unvisited for v in V}
  S := []
  output := []
  for v in V:
    if status[v] = Unvisited:
      let (_, _, sccs) = tarjanSCCstep(v)
      output := output ++ sccs

function tarjanSCCstep(v):
  status[v] = Onstack(i)
  let iv = i
  i := i + 1
  S_local := [v]
  output_local := []

  lowlink = foldl min iv [
    case status[w] of
      Unvisited →
        let (j, S', output') = tarjanSCCstep(w)
        S_local := S_local ++ S'
        output_local := output_local ++ output'
        j
      Onstack(i) → i
      Completed → ∞
    | w in E[v]]

  if lowlink = iv:
    for w in S_local:
      status[w] := Completed
    # finish this SCC
    return (lowlink, [], S_local : output_local)
```

```

else:
    return (lowlink, S_local, output_local)

return output

```

This has two flaws:

1. We have reintroduced local state for `S_local` and `output_local`
2. We now pay the cost of repeated concatenation as we manipulate `S` and output values.

We can defer the concatenations: they also happen along the DFS tree that our algorithm visits. We can just directly represent this tree, and have a separate way to concatenate it...

```

data Tree a = Root a [Tree a]

flatten :: Tree a -> [a]
flatten t = go t []
  where
    go (Root a cs) acc = a : foldr go acc cs

```

A Haskell version, using trees

```

data Status = OnStack Natural | Complete
-- use Nothing as the equivalent of Unvisited from before
type StatusMap k = Map k Status

-- Each node is labelled with
type DFSTree k = Tree (k, [[k]])
data Progress = OnStack Natural | Complete

complete : Y k -> ([k], X k)
incomplete : Y k -> X k
init : Natural -> Y k
act : X k -> Y k -> Y k

-- `graph` is the equivalent of E from before
tarjanSCCStep :: (Ord k) => (k -> [k]) -> k -> State (Natural, Map k Progress)
(SCCNode k)
tarjanSCCStep graph v = do
  i <- state (\(i, m) -> (i, (i + 1, Map.insert v (OnStack i) m)))
  ns <- for (graph v) \w ->
    gets (Map.lookup w . snd) >>= \case
      Nothing -> tarjanStep graph w
      Just (OnStack j) -> pure (SCCNode (Min j) EmptyD EmptyD)
      Just Complete -> pure (SCCNode (Min i) EmptyD EmptyD)
  let n@(SCCNode (Min d) ks kss) = foldl' (<>) (SCCNode (Min i) (ConsD v EmptyD)
EmptyD) ns
  if d == i
  then do
    let ks' = toListD ks
    for_ ks' (\w -> modify (second (Map.insert w Complete)))
    pure $ SCCNode (Min d) EmptyD (ConsD ks' kss)
  else pure n

tarjanSCC :: (Ord k) => (k -> [k]) -> k -> [[k]]
tarjanSCC graph v = toListD $ (\(SCCNode _ _ kss) -> kss) $ evalState (tarjanStep
graph v) (0, mempty)

tarjanSCC' :: (Ord k) => (k -> [k]) -> [k] -> [[k]]

```

```

tarjanSCC' graph vs =
  toListD $
    fold $
      evalState
        ( for vs $ \v ->
          gets (Map.lookup v . snd) >>= \case
            Nothing -> tarjanStep graph v <&> (\(SCCNode _ _ kss) -> kss)
            Just _ -> pure EmptyD
        )
      (0, mempty)

```

Bibliographic notes

The ‘path-based SCC’ is very similar to Tarjan’s, just that instead of storing lowlink numbers, it has a further stack which is popped until the item at the top has the right index. One can think of this as eagerly computing the action of the lowlink values a separate stack.

<https://home.cs.colorado.edu/~hal/Papers/DFS/pbDFShistory.html>

Also, the idea of using Union-Find goes at least as far back to Ian Munro in 1971. This case of union-find can be done in linear-time, as shown by Gabow and Tarjan in *A linear-time algorithm for a special case of disjoint set union*.